DOCKER

Docker is an open-source platform that packages applications and their dependencies into lightweight, portable containers, ensuring they run consistently across different environments. It enables developers to build, ship, and run applications easily without dependency conflicts. Install it on Windows using **Docker Desktop** with WSL 2.

What is Docker?

Docker allows developers to package an application with all of its requirements (libraries, dependencies) into a single unit called a **container**.

- **Containers:** Lightweight, standalone, executable packages of software.
- **Images:** Read-only templates used to create containers.
- **Docker Desktop:** An easy-to-install application for Windows that provides a GUI and tools to manage containers, images, and volumes.

Key Advantages of Docker

- **Portability:** Applications run consistently across different environments, from local machines to cloud platforms like AWS.
- **Efficiency:** By sharing the host OS kernel, containers are faster and use fewer resources than traditional virtual machines.
- **Speed:** Containers start in seconds, speeding up deployment and supporting efficient CI/CD pipelines.
- **Isolation & Security:** Each container runs in an isolated environment, reducing the risk of failures affecting other parts of the system.
- **Consistency:** A standardized environment ensures applications behave the same throughout the development lifecycle.
- **Scalability & Management:** Docker simplifies scaling and, when combined with tools like Kubernetes, enables efficient management of complex, modular microservices

How to Install Docker on Windows

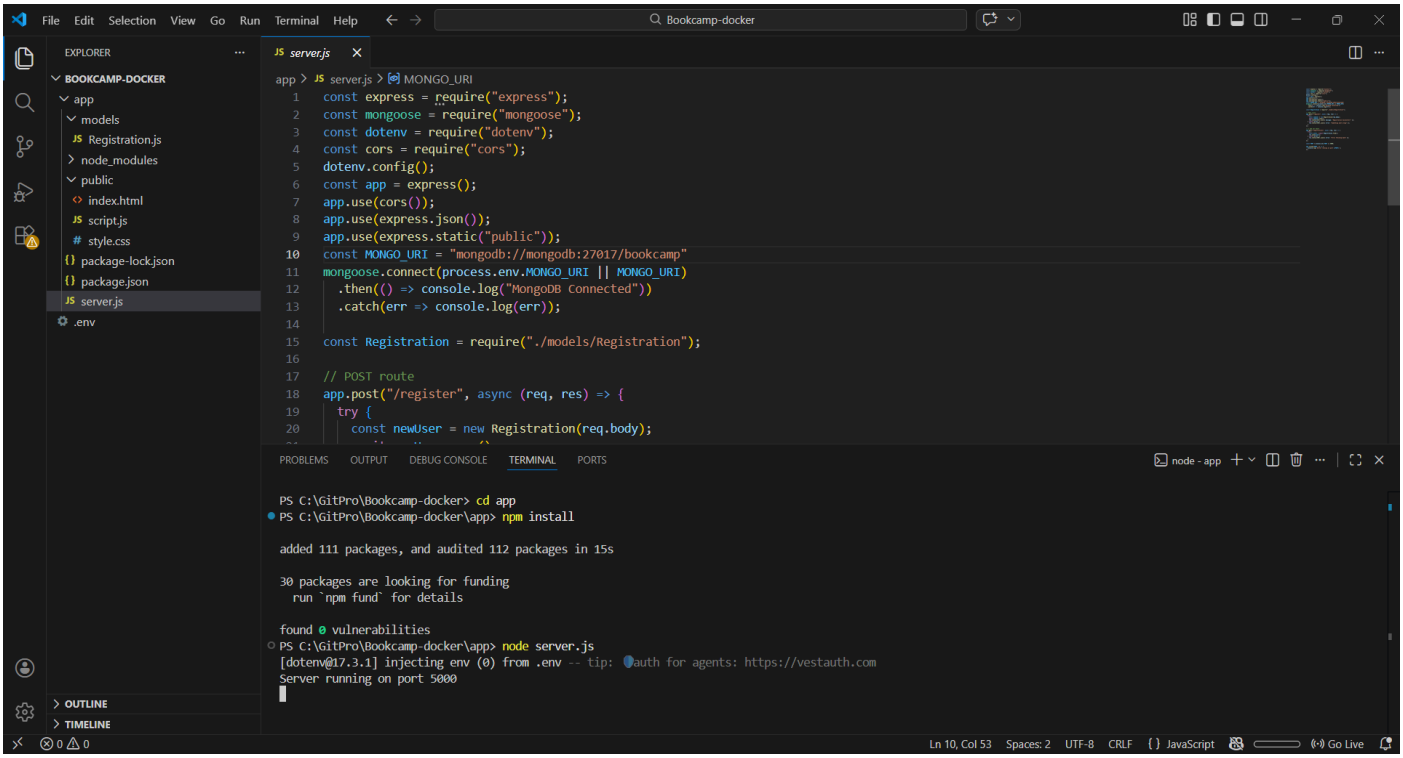
Installation Steps:

1. **Download:** Visit the Docker website and download **Docker Desktop for Windows** (choose AMD64 or ARM64 based on your system).
2. **Install:** Run the .exe installer. During installation, ensure the "Use WSL 2 instead of Hyper-V" option is checked for better performance.
3. **Restart:** After the installation completes, click "Close and restart" to reboot your computer.
4. **Launch:** Open Docker Desktop from the Start menu. Accept the subscription agreement.
5. **Verify:** Open Command Prompt or PowerShell and run docker --version to confirm installation.

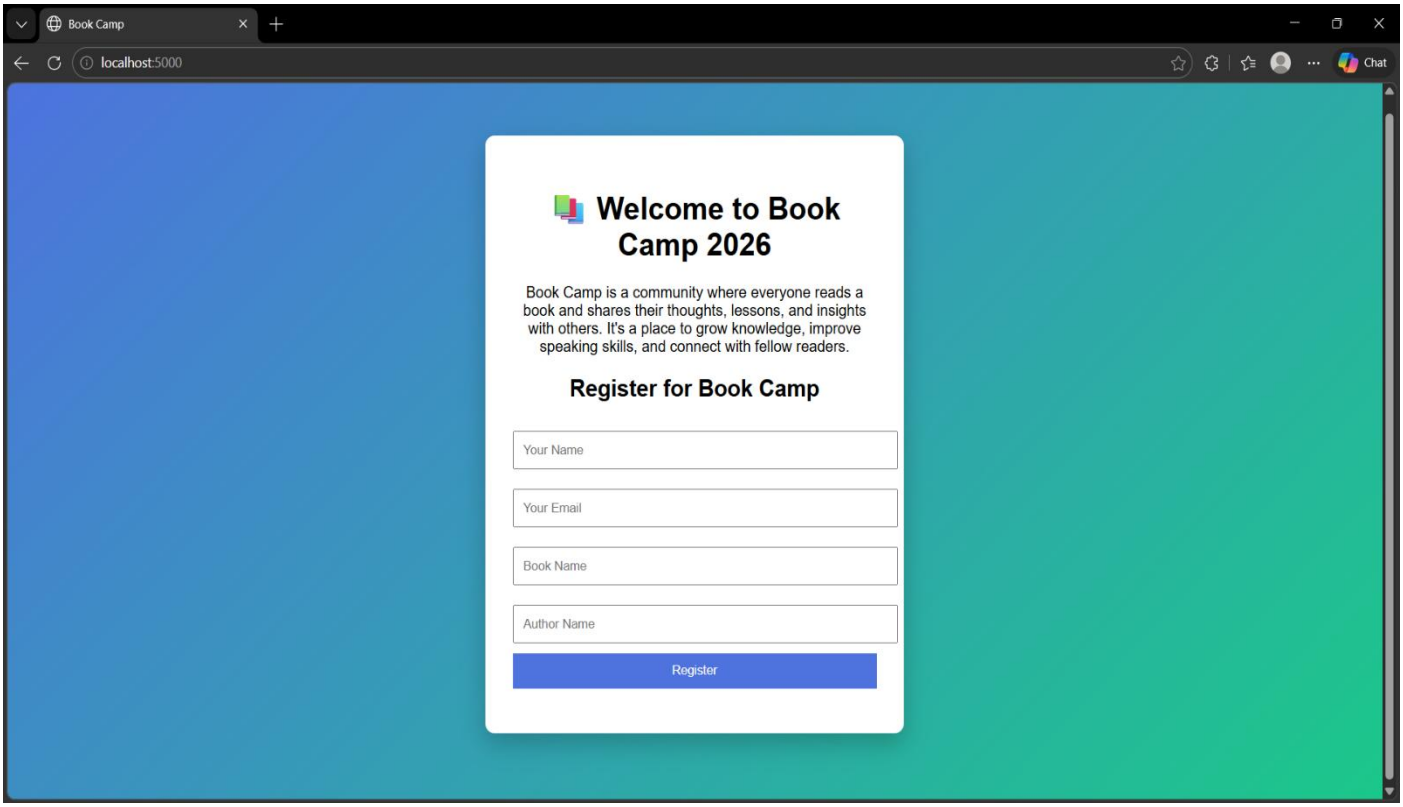
BOOKCAMP_APPLICATION_DOCKERIZATION

STEP1: Clone the repository from https://github.com/maniSource-code/DOCKER.git using - “git clone https://github.com/maniSource-code/DOCKER.git”
Clone it in “C:\Folder” location
Then open visual studio (VS Code) and open this repo folder
Now open terminal in vscode and run these commands

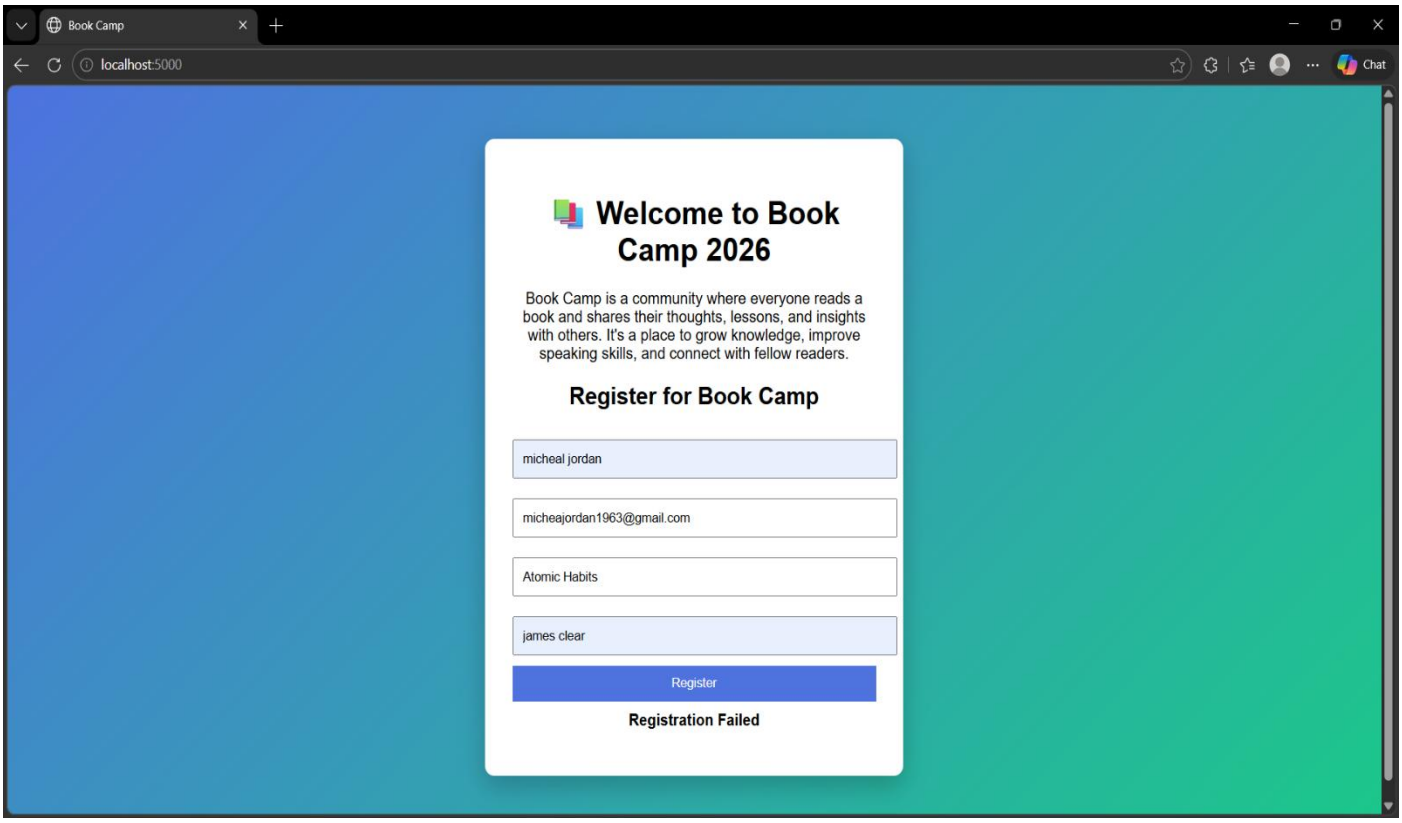
→cd app #changes to app directory
→npm install #installs all npm packages and node_modules
→node server.js #starts the node server you can also use – “npm start”



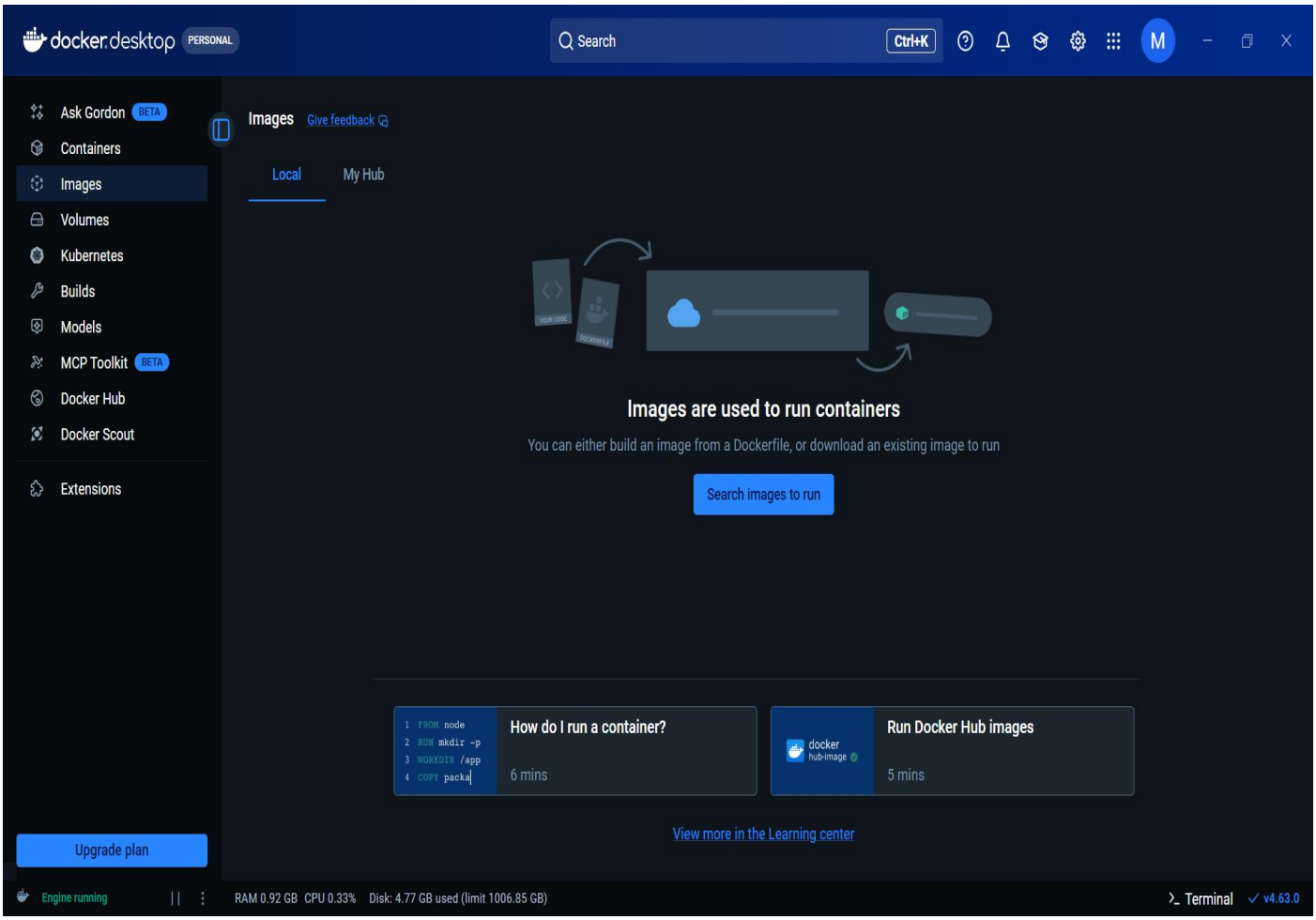
Now the application is running on port 5000 , Browse for http://localhost:5000



Now try to register → Fill the form and click on Register
You will get a message “Registration Failed” that’s because we didn’t connect our Node.js application with any backend database for this now here we will use MongoDB as a backend database using docker



First we need to install and run docker (installation steps are given in page1) , to run docker in windows just open Docker Desktop, it will look like this



Now create a Dockerfile and run these commands

→cd .. #changes directory to parent directory

→docker --version #check docker version

→docker images #lists docker images

→docker build -t app:1.0 . #builds the image naming it app with 1.0 tag

```
FROM node:18

# this WORKDIR /app creates & changes directory to app directory in container
WORKDIR /app

#copies Bookcamp-docker/app json files to /app dir in container
COPY app/package*.json ./

#installs node package manager using package.json file
RUN npm install

#copies all files inside Bookcamp-docker/app dir to /app dir in container
COPY app .

#exposes port 5000
EXPOSE 5000

#runs the node-server
CMD ["node", "server.js"]
```

```
PS C:\GitPro\Bookcamp-docker\app> cd ..
PS C:\GitPro\Bookcamp-docker\app> cd ..
PS C:\GitPro\Bookcamp-docker> docker --version
Docker version 29.2.1, build a5c7197
PS C:\GitPro\Bookcamp-docker> docker images

IMAGE   ID            DISK USAGE   CONTENT SIZE   EXTRA
PS C:\GitPro\Bookcamp-docker> docker build -t app:1.0 .
[+] Building 4.9s (11/11) FINISHED                                docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.1s
 => => transferring dockerfile: 495B                                                0.0s
 => [internal] load metadata for docker.io/library/node:18                          2.6s
 => [auth] library/node:pull token for registry-1.docker.io                         0.0s
 => [internal] load .dockerignore                                                   0.1s
 => => transferring context: 124B                                                   0.0s
 => [internal] load build context                                                   0.5s
```

This builds an app named image with 1.0 tag

Verify if image is created or not using

→docker images #lists images

→docker ps -a #lists all containers

```
node_modules
.git
.gitignore
.env
Dockerfile
Docker-compose.yaml
.dockerignore
```

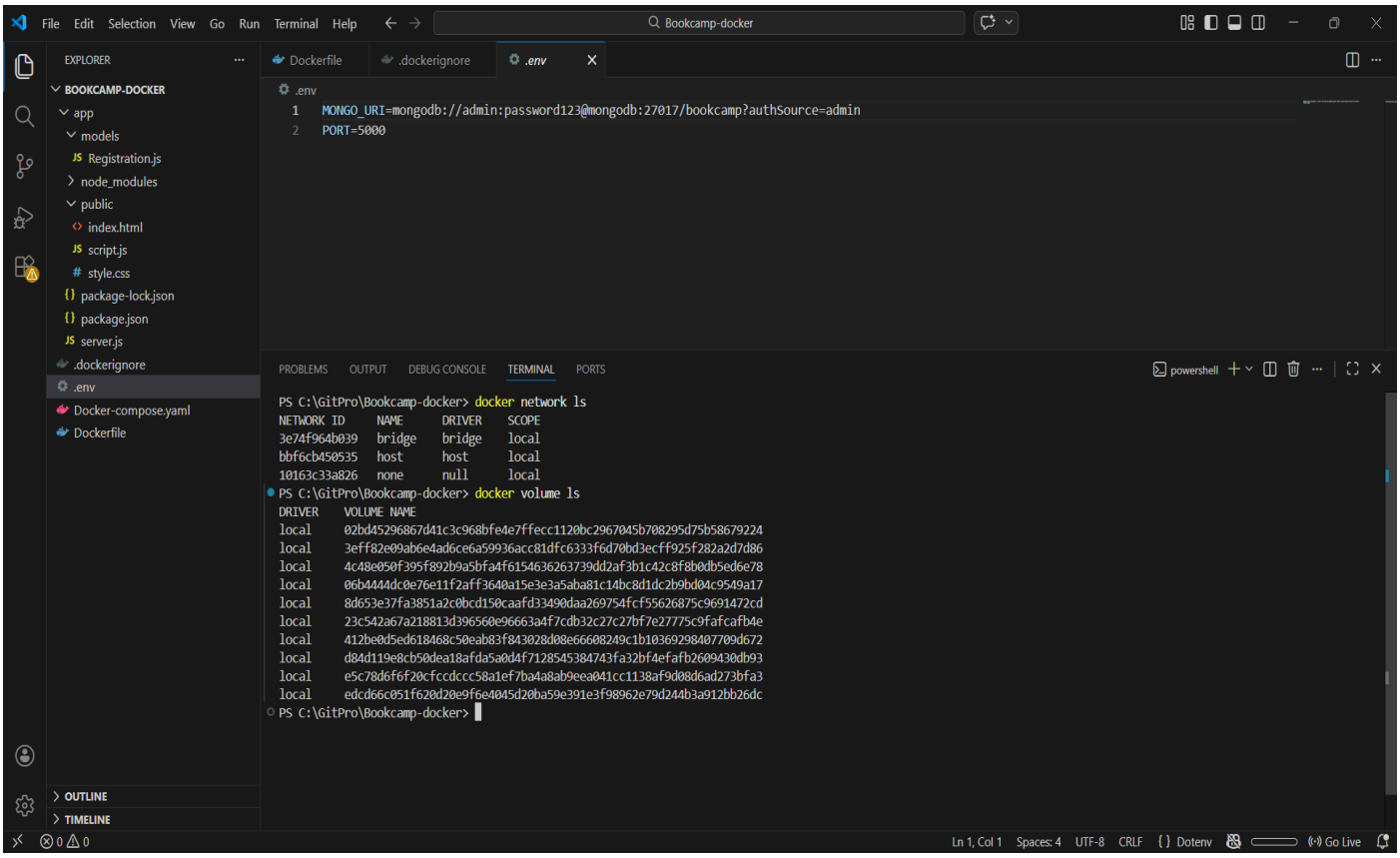
```
PS C:\GitPro\Bookcamp-docker> docker images

IMAGE     ID            DISK USAGE   CONTENT SIZE   EXTRA
app:1.0   462278a74cd7     1.62GB         404MB
PS C:\GitPro\Bookcamp-docker> docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS   PORTS   NAMES
PS C:\GitPro\Bookcamp-docker>
```

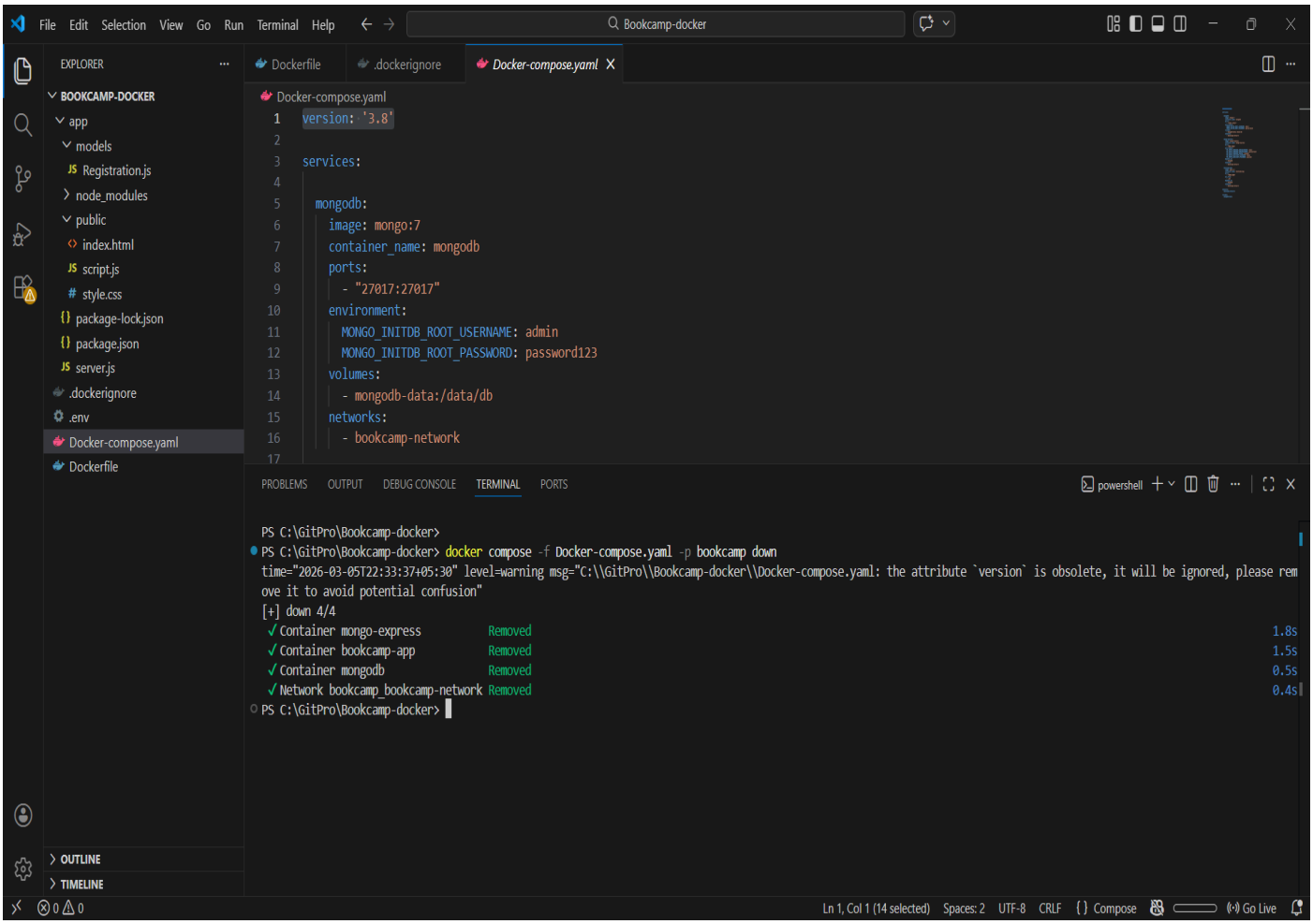
Now create a Docker-compose.yaml file and run thse in terminal

→docker network ls #lists all Docker Networks

→docker volume ls #lists all Docker Volumes



Run →docker compose -f Docker-compose.yaml -p bookcamp



Verify if images are created and containers are running using

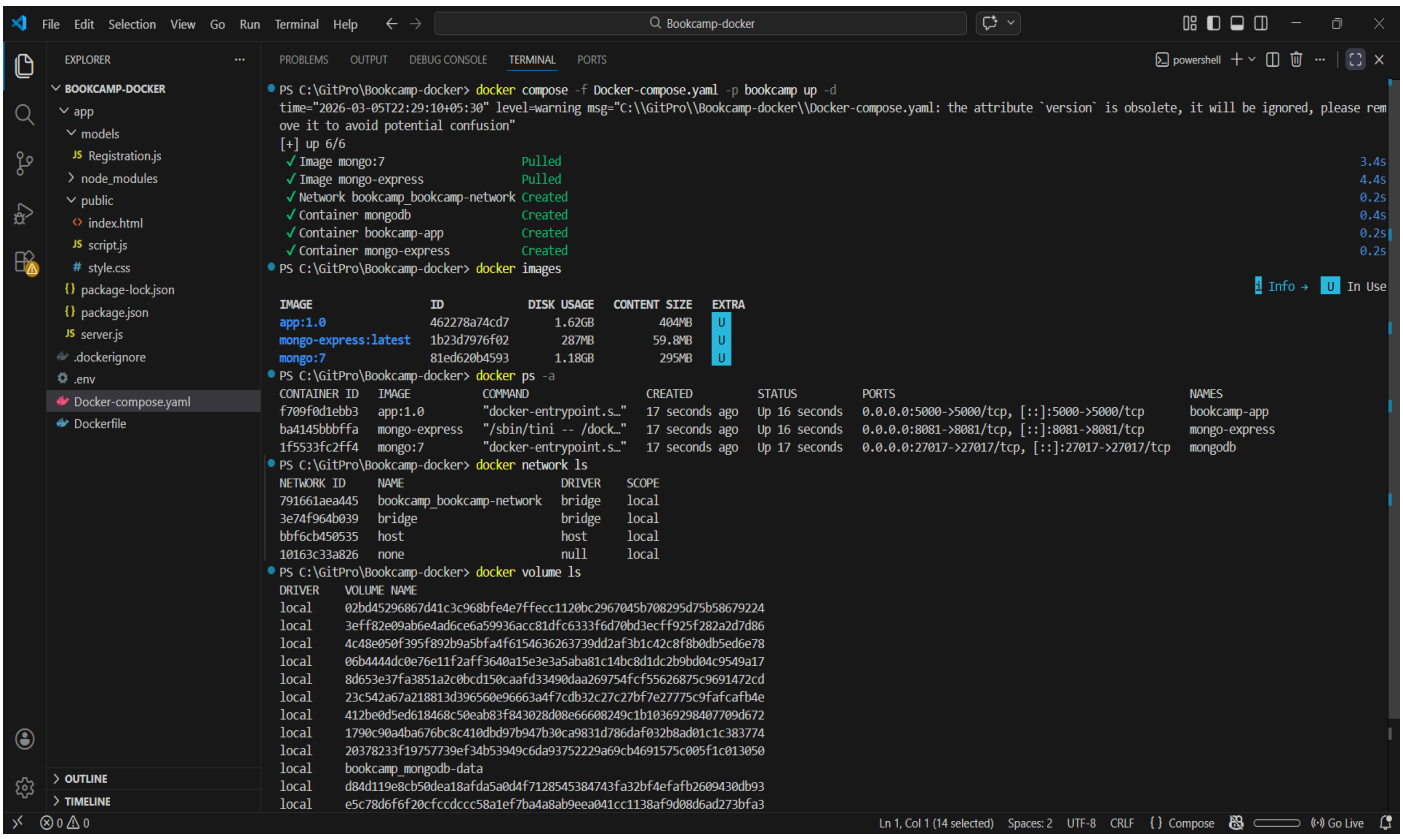
→docker images

→docker ps

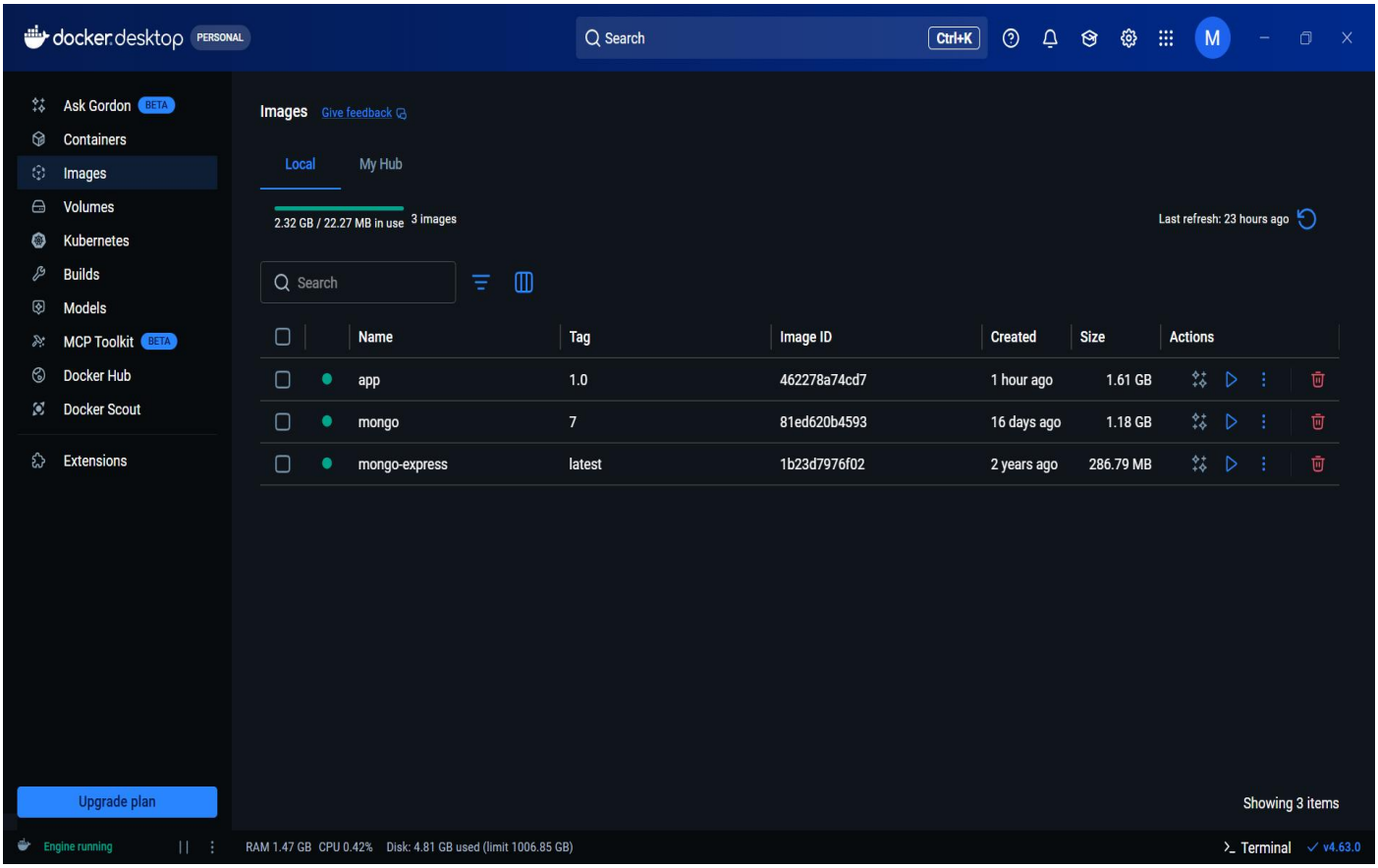
And also check that network and volumes are also created using docker compose or not, Use

→docker network ls

→docker volume ls



OR you can verify if images and containers created and running in Docker Desktop, it would look like this



docker desktop PERSONAL

Search Ctrl+K

Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions

Upgrade plan

Engine running

Containers

Give feedback

Container CPU usage
1.83% / 1200% (12 CPUs available)

Container memory usage
243.22MB / 3.58GB

Show charts

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usag...	Memory (%)	Disk rea	Actions
	bookcamp	-	-	-	1.83%	243.22MB / 10.9	6.48%	0B / 307	

Showing 1 item

RAM 1.51 GB CPU 0.08% Disk: 4.81 GB used (limit 1006.85 GB)

Terminal v4.63.0

docker desktop PERSONAL

Search Ctrl+K

Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions

Upgrade plan

Engine running

bookcamp

C:\GitPro\Bookcamp-docker

View configurations

mongodb
mongo:7
27017:27017

bookcamp-app
app:1.0
5000:5000

mongo-expre...
mongo-express
8081:8081

```
WSL2"},"platform":"Node.js v18.20.3, LE (unified)|Node.js v18.20.3, LE (unified)"}}}
{"t":{"$date":"2026-03-05T17:05:21.443+00:00"},"s":"I",  "c":"ACCESS",   "id":6788604,
"ctx":"conn5","msg":"Auth metrics report","attr":{"metric":"acquireUser","micros":0}}
{"t":{"$date":"2026-03-05T17:05:21.461+00:00"},"s":"I",  "c":"ACCESS",   "id":5286306,
"ctx":"conn5","msg":"Successfully authenticated","attr":
{"client":"172.18.0.3:41722","isSpeculative":true,"isClusterMember":false,"mechanism":"SCRAM-
SHA-256","user":"admin","db":"admin","result":0,"metrics":{"conversation_duration":
{"micros":18166,"summary":{"0":{"step":1,"step_total":2,"duration_micros":217},"1":
{"step":2,"step_total":2,"duration_micros":183}}}},"doc":{"driver":
{"name":"nodejs","version":"4.13.0"},"os":
{"type":"Linux","name":"linux","architecture":"x64","version":"6.6.87.2-microsoft-standard-
WSL2"},"platform":"Node.js v18.20.3, LE (unified)|Node.js v18.20.3, LE (unified)"},"extraInfo":
{}}}
{"t":{"$date":"2026-03-05T17:05:21.463+00:00"},"s":"I",  "c":"NETWORK",  "id":6788700,
"ctx":"conn5","msg":"Received first command on ingress connection since session start or auth
handshake","attr":{"elapsedMillis":2}}
```

mongo-express

```
Mongo Express server listening at http://0.0.0.0:8081
Server is open to allow connections from anyone (0.0.0.0)
```

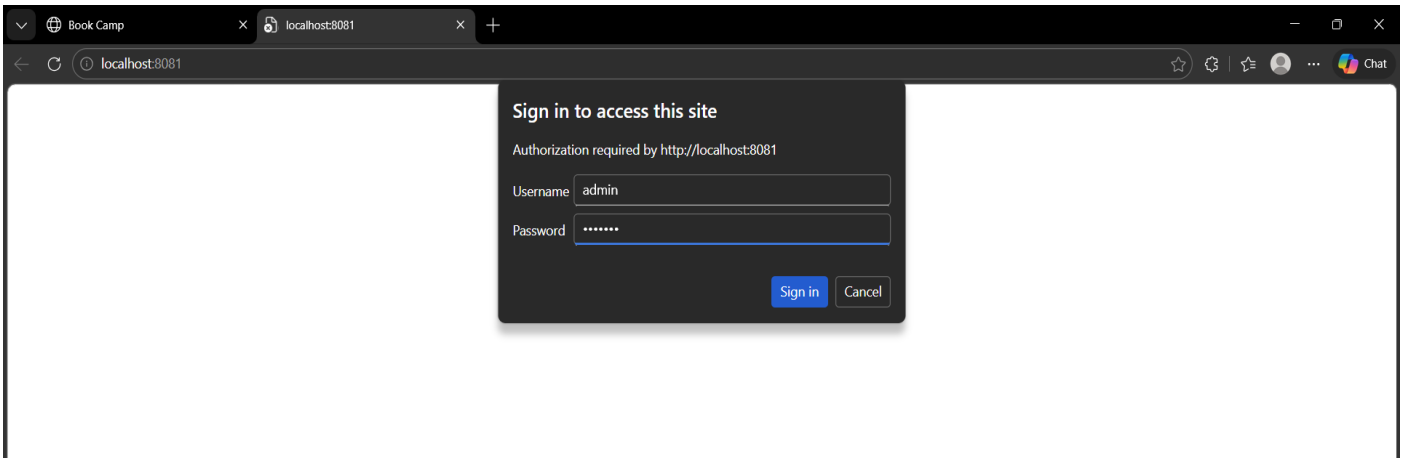
mongodb

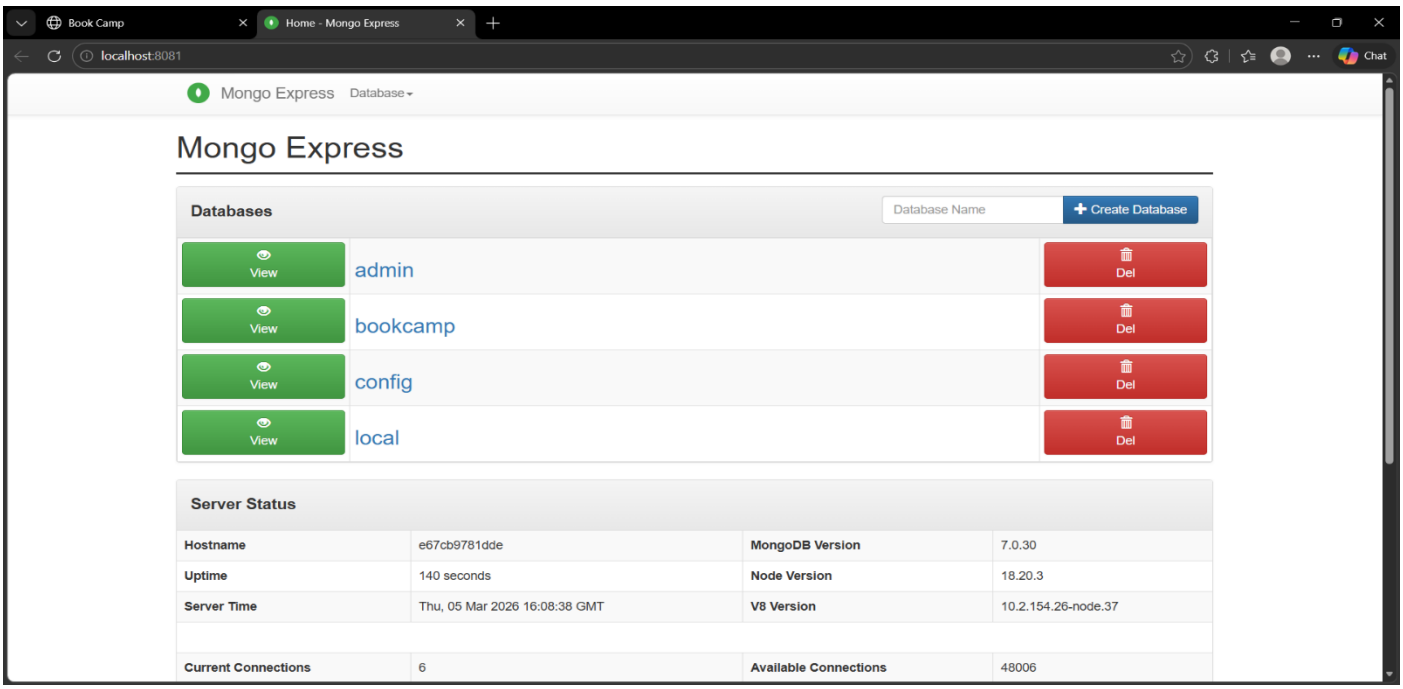
```
{"t":{"$date":"2026-03-05T17:05:29.588+00:00"},"s":"I",  "c":"WTCHKPT",  "id":22430,
"ctx":"Checkpointer","msg":"WiredTiger message","attr":{"message":
{"ts_sec":1772730329,"ts_usec":588294,"thread":"1:0x7012b36e4640","session_name":"WT_SESSION.ch
eckpoint","category":"WT_VERB_CHECKPOINT_PROGRESS","category_id":6,"verbose_level":"DEBUG_1","v
erbose_level_id":1,"msg":"saving checkpoint snapshot min: 4, snapshot max: 4 snapshot count: 0,
oldest timestamp: (0, 0) , meta checkpoint timestamp: (0, 0) base write gen: 40"}}}
{"t":{"$date":"2026-03-05T17:05:31.942+00:00"},"s":"I",  "c":"NETWORK",  "id":22943,
"ctx":"listener","msg":"Connection accepted","attr":
{"remote":"172.18.0.3:37838","isLoadBalanced":false,"uuid":{"uuid":{"$uuid":"6ce69f38-ff15-
4d2c-82bd-f1e656d3c269"}},"connectionId":6,"connectionCount":6}}
{"t":{"$date":"2026-03-05T17:05:31.946+00:00"},"s":"I",  "c":"NETWORK",  "id":51800,
```

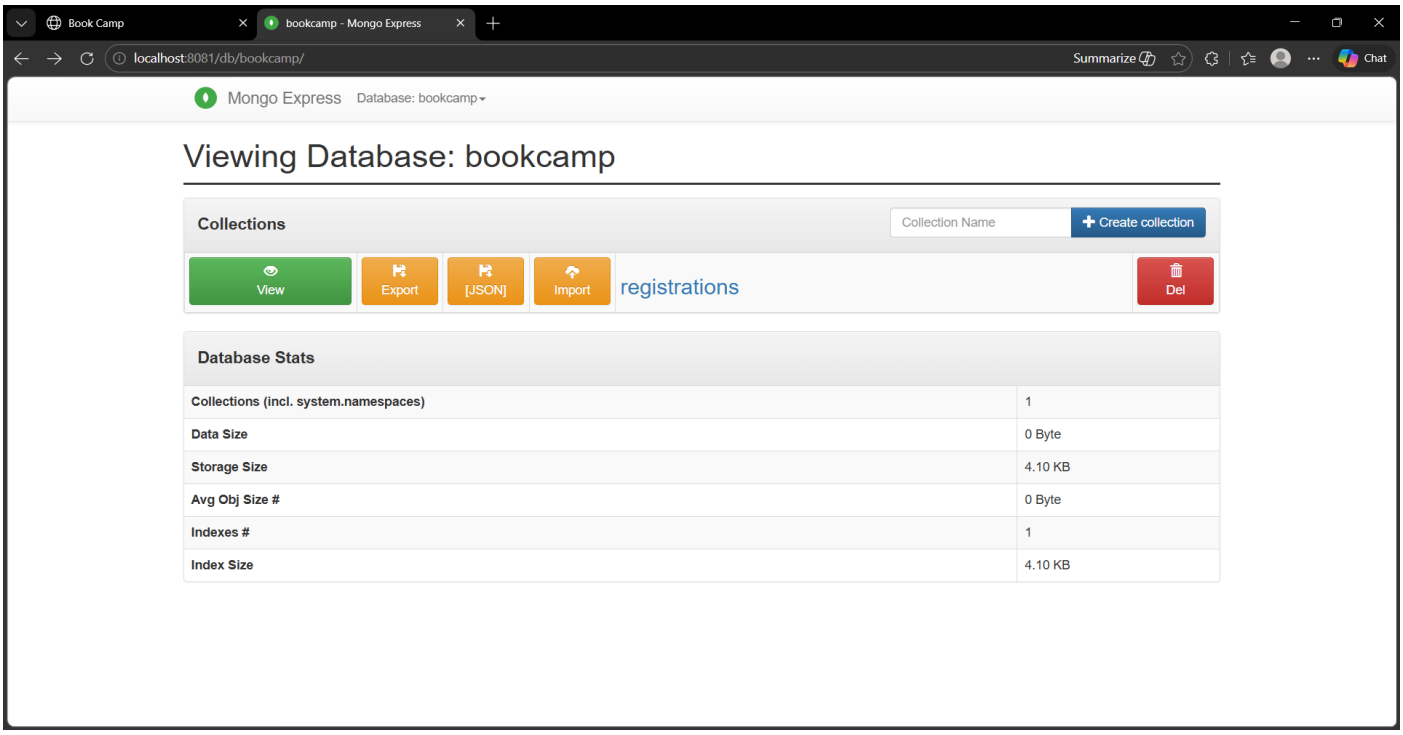
RAM 1.51 GB CPU 0.08% Disk: 4.81 GB used (limit 1006.85 GB)

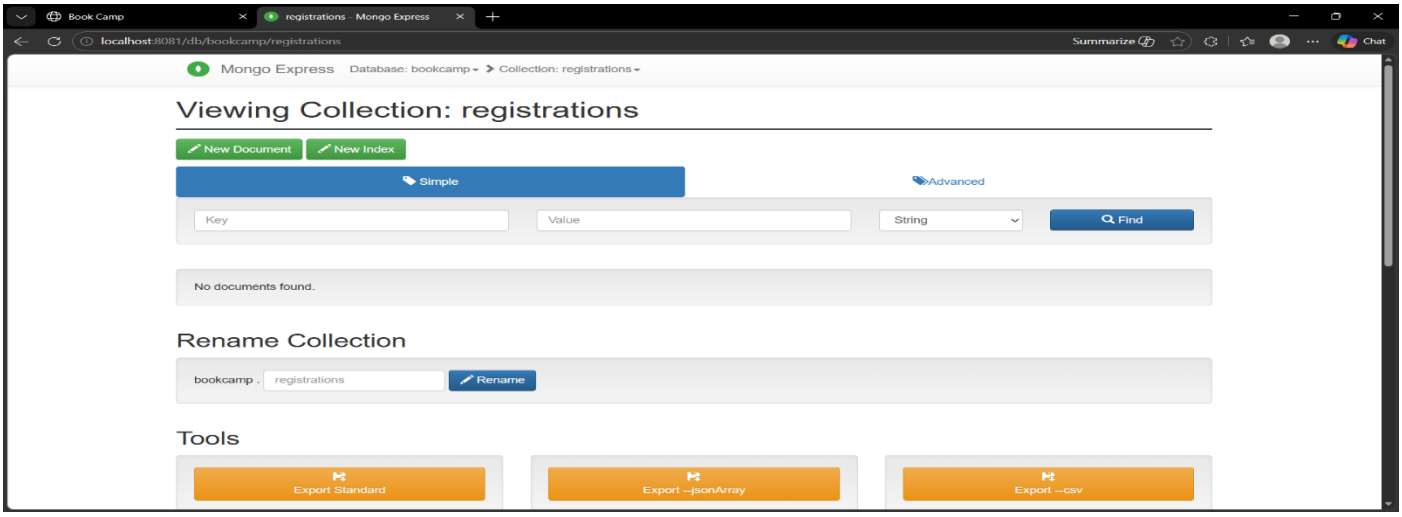
Terminal v4.63.0

Now browse http://localhost:5000 for our Node.js (backend app) server and http://localhost:8081 for the Mongo-Express (Ui for Mongodb)
Login with credentials set in environment variables for mongo-express service in docker compose file

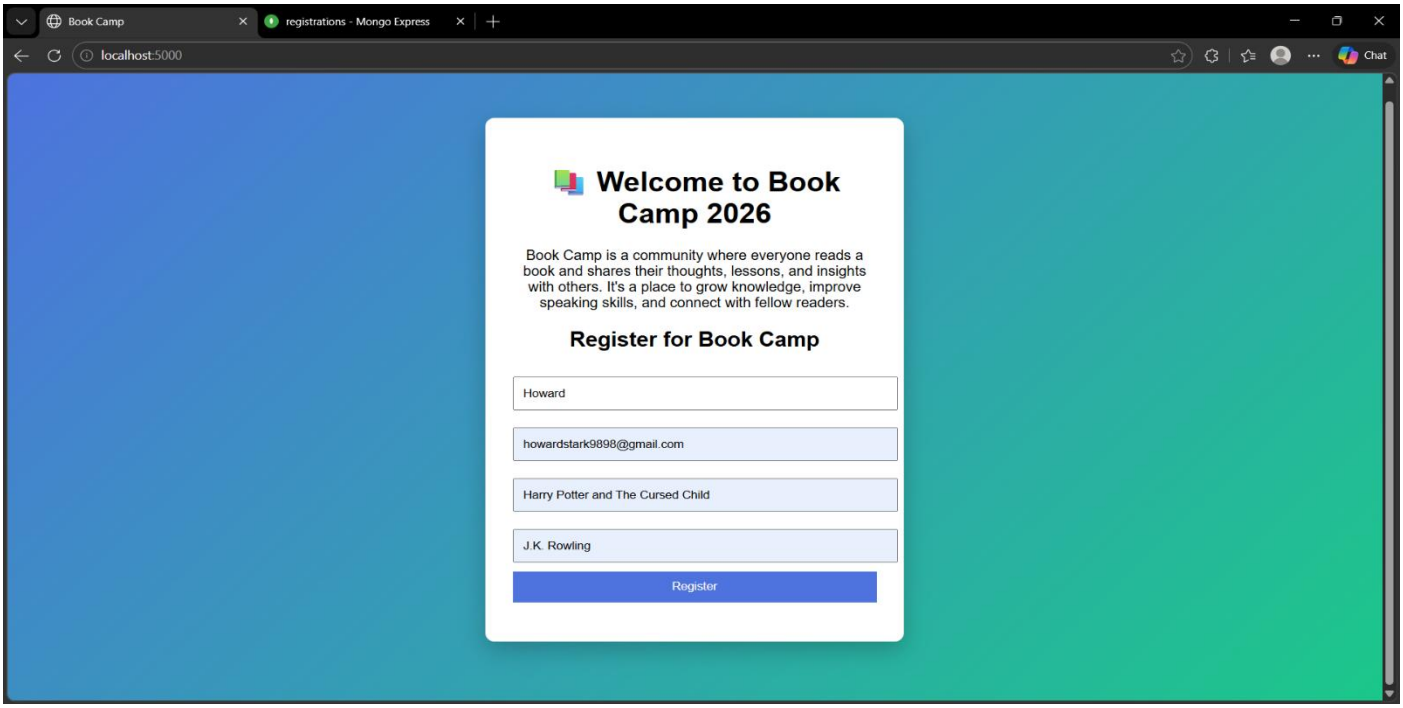


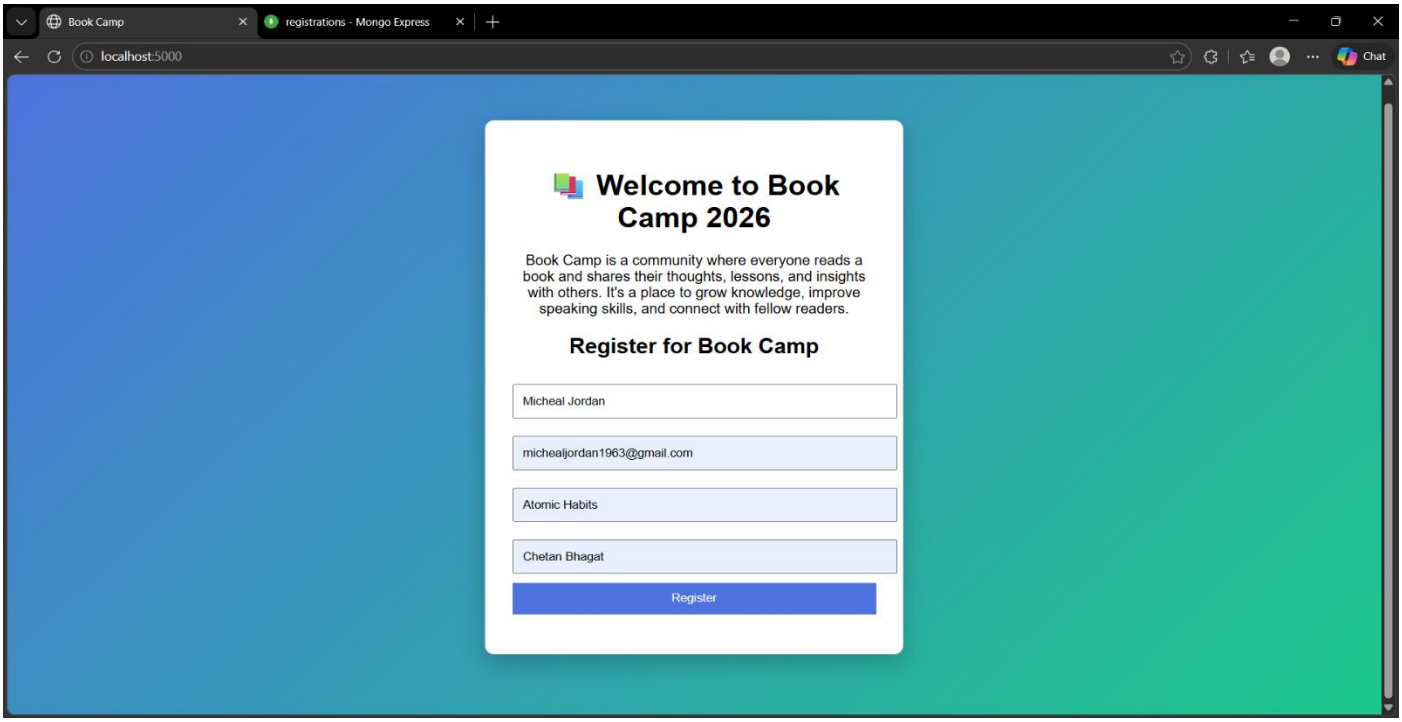


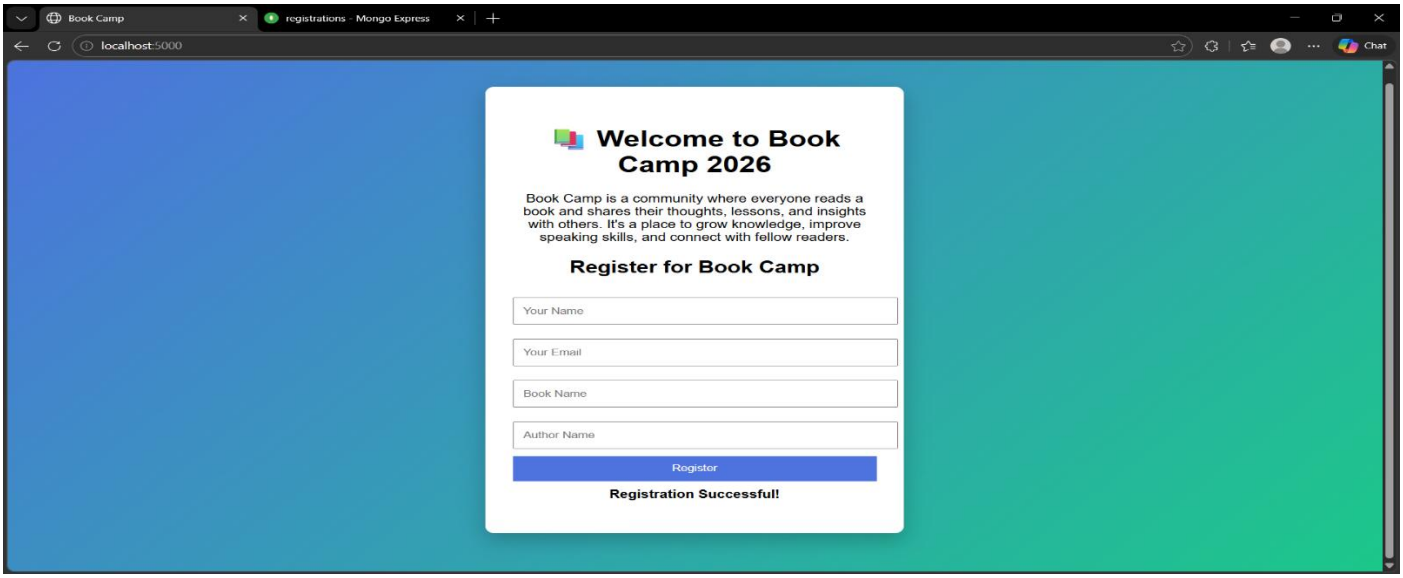




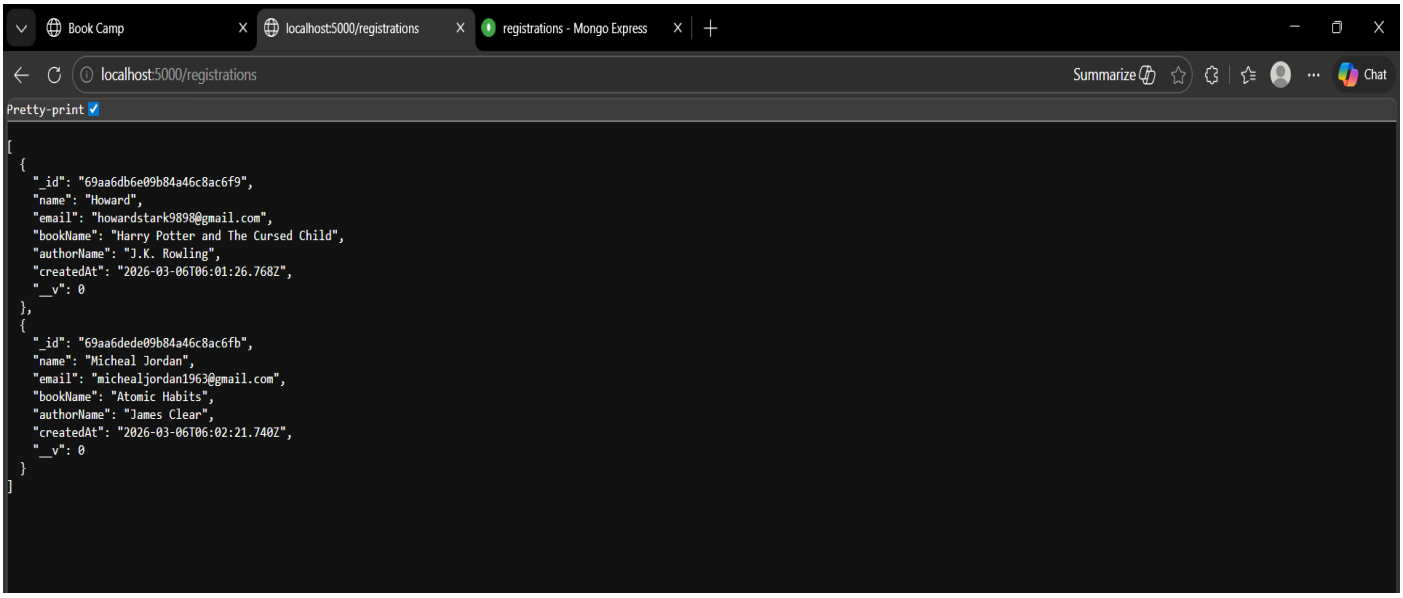
Now try to register now →Fill the form and click on Register
You will get a message "Registration Successful"



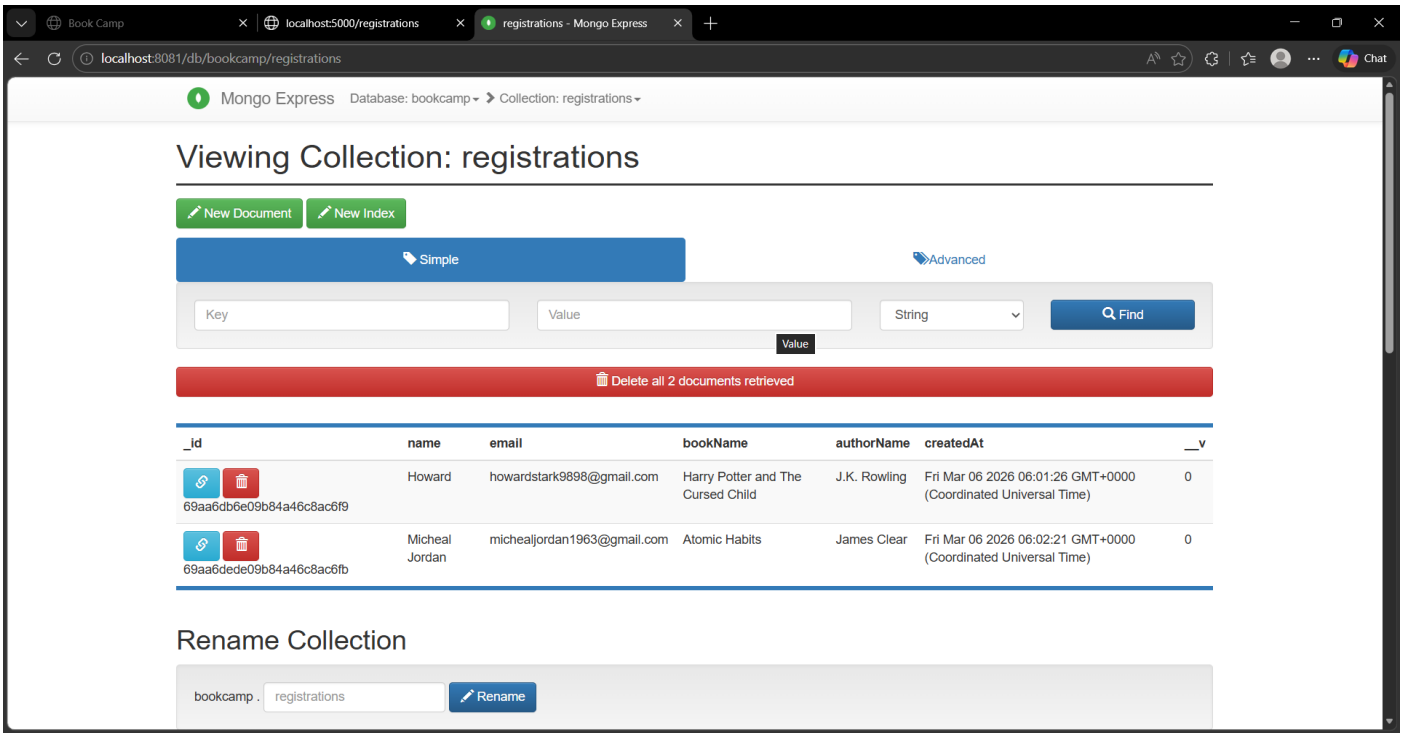




Now visit http://localhost:5000/registrations this page shows data in JSON format.



Check the registration table in bookcamp database in Mongo-Express, it will look like this

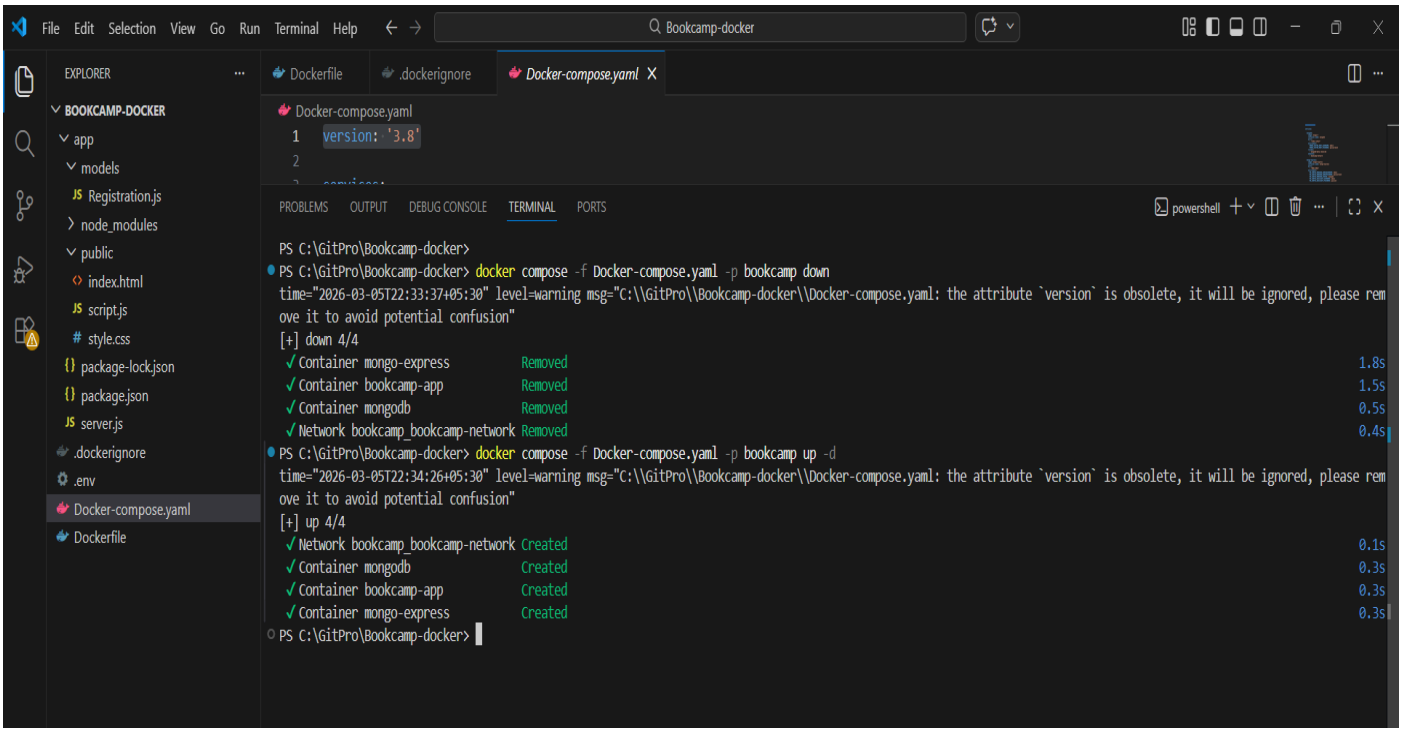


Now even if you delete the containers the data will be stored In locally because we used docker volumes in docker compose file itself so if you rebuild the containers again them the data will be persisted meaning data is not lost even if containers are deleted

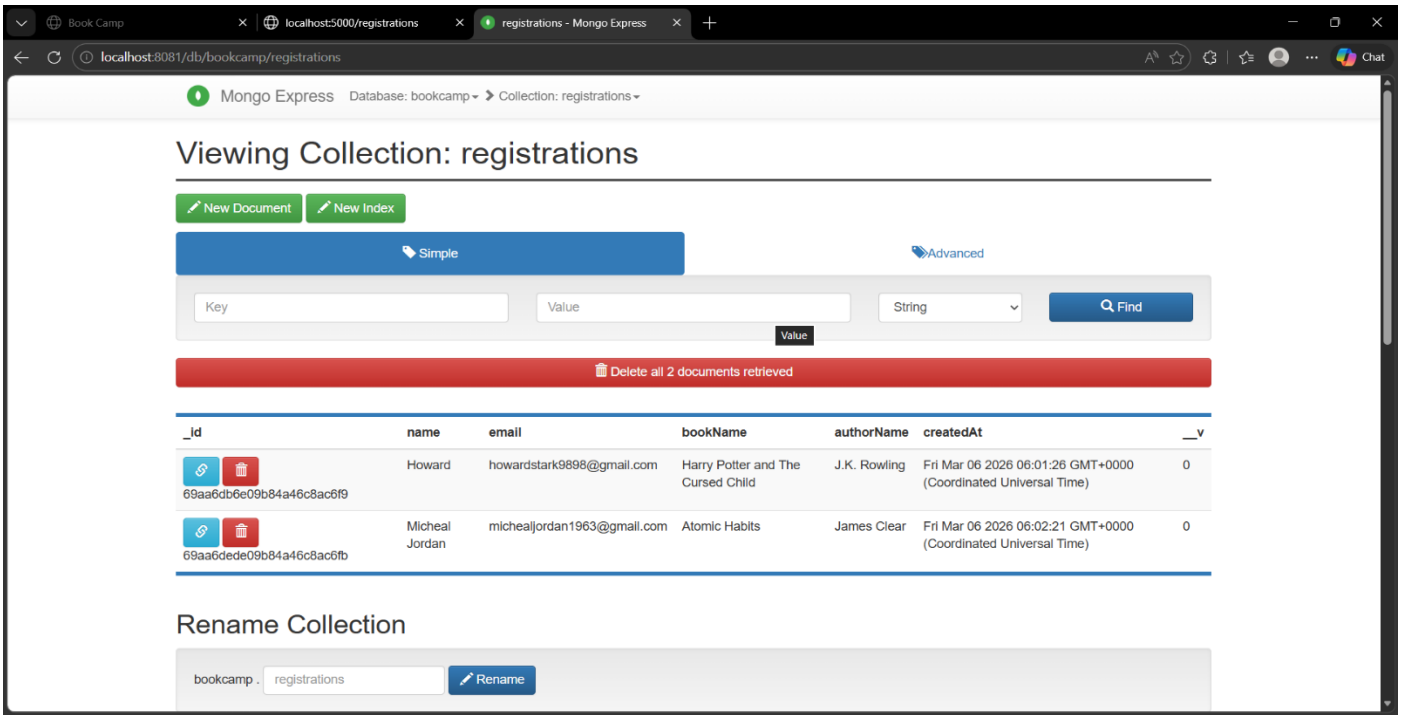
To check this use

→docker compose -f Docker-compose.yaml -p bookcamp down #deletes the containers

→docker compose -f Docker-compose.yaml -p bookcamp up -d #creates the containers and runs them



Now check the registration table in Mongo-Express, you will find the previous data still intact.

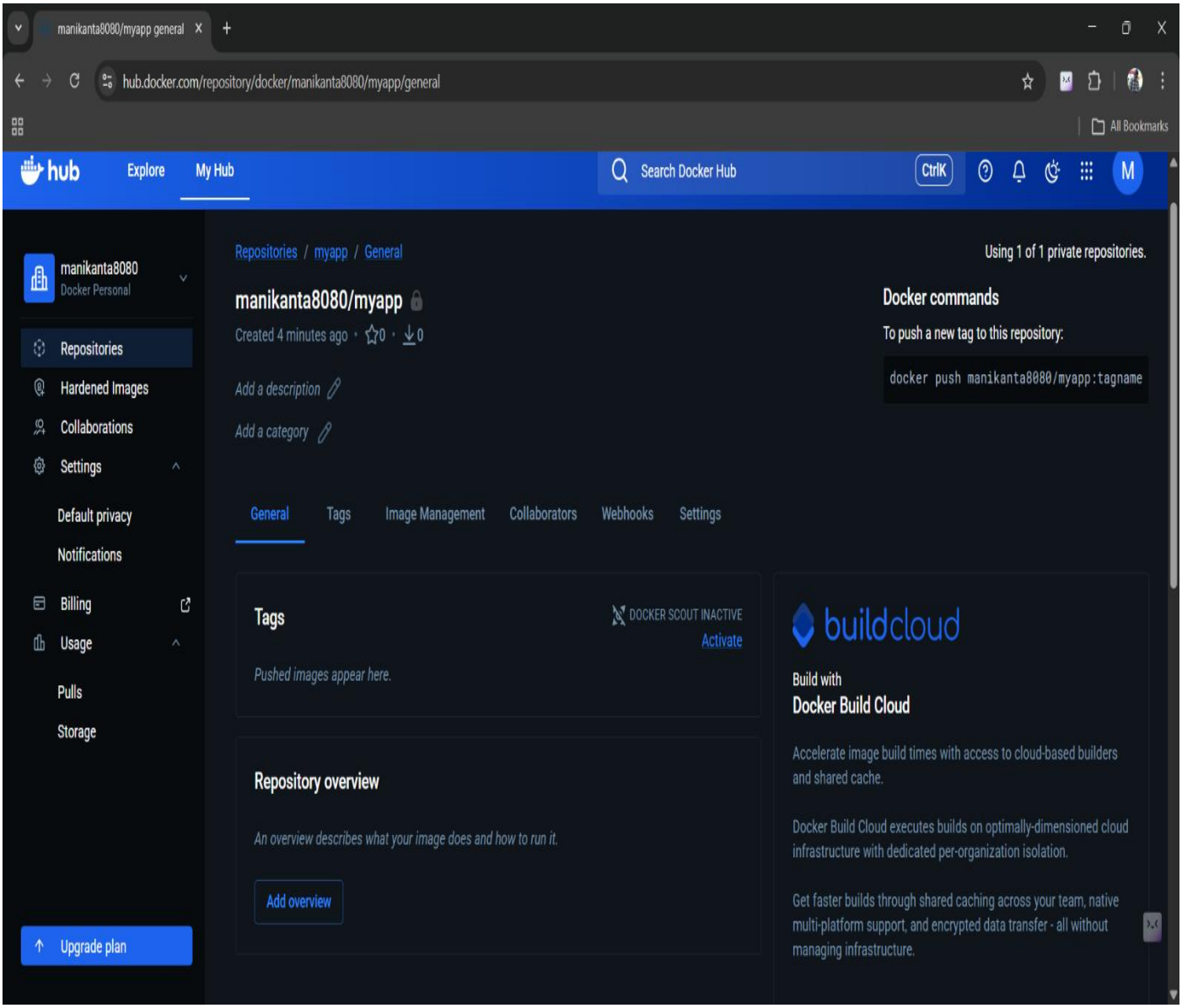


DOCKER REPOSITORY

For development and deployment we can use many Repositories like AWS-ECR, Docker Hub Repository or a Private repository-(only the organisation will have access) to store the images, then we can pull these images later.

Here we will see how to push images into Docker Hub repository

1. First create a Docker Hub account then,go to repositories and create a repository(private or public)
   Example: repository name: myapp
   Note: docker allows to create 1 private repository but you can create many public repositories
2. Go to command line interface and login with docker using →docker login- Enter username and password
3. Then use command →docker images - to verify or list images
   Example: app:1.0
4. Now tag the image with Docker Hub using →docker tag <image> <username>/<repository>:<tag>
   Example: docker tag app:1.0 johnpaul/myapp:1.0.0
5. Push the image into DockerHub repository using → docker push <username>/<repository>:<tag>
   Example:docker push Johnpaul/myapp:1.0.0
6. Now check the DockerHub myapp Repository you will see the tagged image there



Administrator: Command Prompt

```
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>docker login
Authenticating with existing credentials... [Username: manikanta8080]

i Info → To login with a different account, run 'docker logout' followed by 'docker login'


Login Succeeded

C:\Windows\System32>docker images

                                                                                                  i Info →   U   In
Use
IMAGE                    ID             DISK USAGE   CONTENT SIZE   EXTRA
app:1.0                  462278a74cd7       1.62GB          404MB    U
mongo-express:latest     1b23d7976f02        287MB         59.8MB    U
mongo:7                  81ed620b4593       1.18GB          295MB    U

C:\Windows\System32>docker tag app:1.0 manikanta8080/myapp:1.0.0

C:\Windows\System32>docker images

                                                                                                  i Info →   U   In
Use
IMAGE                      ID             DISK USAGE   CONTENT SIZE   EXTRA
app:1.0                    462278a74cd7       1.62GB          404MB    U
manikanta8080/myapp:1.0.0  462278a74cd7       1.62GB          404MB    U
mongo-express:latest       1b23d7976f02        287MB         59.8MB    U
mongo:7                    81ed620b4593       1.18GB          295MB    U

C:\Windows\System32>docker push manikanta8080/myapp:1.0.0
The push refers to repository [docker.io/manikanta8080/myapp]
f87f314c30cf: Pushed
79b2f47ad444: Pushed
5fb3ce334ee4: Pushed
3697be50c98b: Pushed
f2afa0a80274: Pushed
c6b30c3f1696: Pushed
48523303d3ea: Pushed
3e6b9d1a9511: Pushed
37927ed901b1: Pushed
cda7f44f2bdd: Pushed
57405dfbf362: Pushed
e23f099911d6: Pushed
461077a72fb7: Pushed
1.0.0: digest: sha256:462278a74cd74c8041b33050309a778888feebd1a0b40aa9c8a56402e4911157 size: 856
```

manikanta8080/myapp general

hub.docker.com/repository/docker/manikanta8080/myapp/general

hub Explore My Hub Search Docker Hub CtrlK

manikanta8080
Docker Personal

- Repositories
- Hardened Images
- Collaborations
- Settings
  - Default privacy
  - Notifications
- Billing
- Usage
  - Pulls
  - Storage

Upgrade plan

Repositories / myapp / General

Using 1 of 1 private repositories.

manikanta8080/myapp

Last pushed 2 minutes ago • Repository size: 385.4 MB • 0 • 0

Add a description

Add a category

General Tags Image Management Collaborators Webhooks Settings

Tags

DOCKER SCOUT INACTIVE
Activate

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
1.0.0		Image	less than 1 day	2 minutes

See all

Repository overview

Docker commands

To push a new tag to this repository:

```
docker push manikanta8080/myapp:tagname
```

buildcloud

Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.